

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет по лабораторной работе №2
Тема: «Подсистема прерываний микроконтроллера MSP430F5529»

Выполнил:
студент группы 250541 Власов Р. Е.

Проверил:
к.т.н., доцент _____ Селезнёв И. Л.

Минск
2025

1. ЦЕЛЬ РАБОТЫ

Изучить принципы работы подсистемы прерываний микроконтроллера MSP430F5529.

2. ИСХОДНЫЕ ДАННЫЕ К РАБОТЕ

1. Интегрированная среда разработки Code Composer Studio.
2. Плата MSP-EXP430F5529.
3. Программа должна работать следующим образом:
 - после запуска должны гореть 2 светодиода;
 - после первого нажатия на кнопку – первый светодиод не горит, а второй мигает;
 - после второго нажатия на кнопку – первый мигает, а второй не горит;
 - после третьего нажатия на кнопку – горят оба светодиода;
 - режимы работы должны меняться по отпусканию кнопки;
 - программная обработка дребезга кнопки

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

3.1 Прерывания

Различают системные немаскируемые (SMNI), пользовательские немаскируемые (UNMI) и маскируемые прерывания. К системным немаскируемым относятся: сигнал RST/NMI в режиме NMI, сбой генератора, ошибка доступа Flash памяти. К пользовательским немаскируемым - сбой напряжения питания (от подсистемы PMM), доступ к несуществующей (vacant) памяти, события с буфером (mailslot) JTAG интерфейса. Маскируемые прерывания могут быть отключены (замаскированы) индивидуально или все сразу (бит GIE регистра состояния SR).

Кратко рассмотрим, как происходит обработка прерывания. Задержка от возникновения запроса на прерывание до начала выполнения обработчика составляет 6 циклов. При этом заканчивается выполнение текущей инструкции, счетчик команд PC сохраняется в стеке (указывает на следующую команду), регистр состояния SR сохраняется в стеке, выбирается прерывание с максимальным приоритетом (если поступило несколько запросов), автоматически сбрасывается флаг запроса от отдельного прерывания (сброс общего флага запроса должен осуществляться программно). Далее, все биты SR сбрасываются, за исключением SCG0, так как останавливаются все режимы с низким питанием. Так как бит GIO при этом устанавливается в 0, все прерывания запрещаются. Наконец, вектор (адрес обработчика) загружается в PC.

Из-за конвейерной архитектуры процессора, команда, следующая за EINT (разрешение прерывания), всегда выполняется, даже если запрос на прерывание возник до его разрешения. Если за EINT сразу следует DINT, прерывание, ожидающее обработки может быть не обслужено. Команды, следующие за DINT, в этом случае, могут работать некорректно. Аналогичные последствия вызываются альтернативными командами, которые устанавливают и сразу сбрасывают флаг GIE регистра состояний. Рекомендуется вставлять хотя бы одну команду между EINT и DINT.

Возврат из прерывания выполняется командой RETI, которая выполняется за 5 циклов и загружает из стека SR, PC. Таблица векторов прерываний располагается по адресам 0FFFFh – 0FF80h и содержит 64 вектора. Бит SYSRIVECT регистра SYCTL позволяет определить альтернативную таблицу векторов, в старших адресах RAM. По сигналу сброса этот бит автоматически сбрасывается.

За прерывания отвечают ряд системных регистров (табл. 3.1). Пользовательские маскируемые прерывания рассматриваются отдельно при обсуждении соответствующего функционального узла архитектуры микроконтроллера, в частности, ранее уже рассматривались регистры для работы с прерываниями от цифровых портов ввода-вывода. В табл. 3.2 представлены поля системных регистров для работы с прерываниями.

Таблица 3.1 – Регистры для работы с прерываниями

Регистр	Адрес	Назначение
SFRIE1	0100h	Разрешение прерываний
SFRIFG1	0102h	Флаги прерываний
SYCTL	0180h	Регистр управления
SYSBERRIV	0198h	Генератор вектора ошибок шины
SYSUNIV	019Ah	Генератор вектора пользовательских NMI
SYSSNIV	019Ch	Генератор вектора системных NMI
SYSRSTIV	019Eh	Генератор вектора сброса

Работа с прерываниями достаточно проста. Вначале необходимо разрешить соответствующее прерывание, например, P1IE |= BIT7; - разрешает прерывание по входу 7 вывода порта 1, в экспериментальной плате к нему подключена кнопка S1. После того, как режим инициализирован, хорошим тоном считается перевод контроллера в режим пониженного энергопотребления. Сделать это можно, используя вызов

`__bis_SR_register`, например, следующий фрагмент переводит контроллер в режим LPM0 с разрешением прерываний:

```
__bis_SR_register(LPM0_bits + GIE);
```

Еще одной особенностью запуска в среде отладки Code Composer Studio является необходимость вызова `__no_operation()` перед завершением функции `main`, если она не использует некоторого цикла. Без этого вызова с завершением функции `main` завершится и выполнение кода в оболочке.

Собственно обработчик прерывания описывается с использованием директивы `#pragma vector`. Например, фрагмент кода ниже описывает обработчик прерывания от порта ввода-вывода 1:

```
#pragma vector=PORT1_VECTOR
__interrupt void PORT1_ISR(void)
{...}
```

Таблица 3.2 – Поля регистров для работы с прерываниями

Регистр	Биты	Поле	Назначение
SFRIE1	7	JMBOUTIE	Разрешение прерываний выхода JTAG
	6	JMBINIE	Разрешение прерываний входа JTAG
	5	ACCVIE	Разрешение прерываний нарушения доступа Flash
	4	NMIE	Разрешение прерываний вывода NMI
	3	VMAIE	Разрешение прерываний доступа к несуществующей памяти
	1	OFIE	Разрешение прерываний сбоя генератора
	0	WDTIE	Разрешение прерываний сторожевого таймера
SFRIFG1	7	JMBOUTIFG	Флаг прерывания выхода JTAG
	6	JMBINIFG	Флаг прерывания входа JTAG
	4	NMIIFG	Флаг прерывания NMI
	3	VMAIFG	Флаг прерывания доступа к несуществующей памяти
	1	OFIFG	Флаг прерывания сбоя генератора
	0	WDTIFG	Флаг прерывания сторожевого таймера

Продолжение таблицы 3.2.

SYSCTL	0	SYSRIVECT	Вектор прерывания при выходе за пределы RAM (64К или полностью)
SYSUNIV	0-15	SYSUNIV	Вектор пользовательского NMI
SYSSNIV	0-15	SYSSNIV	Вектор системного NMI
SYSRSTIV	0-15	SYSRSTIV	Вектор прерываний сброса
SYSBERRIV	0-15	SYSBSLOFF	Вектор прерываний ошибки системной шины

3.2 Прерывания от портов ввода-вывода MSP430F5529

Порты ввода-вывода микроконтроллера MSP430F5529 (P1 и P2) могут выступать источниками маскируемых прерываний. Это позволяет реагировать на внешние события (нажатие кнопки, изменение уровня на входе) без постоянного опроса линий в основном цикле программы.

Для каждого из портов P1 и P2 предусмотрен набор специальных регистров:

- 1) PxIE (Interrupt Enable) – регистр разрешения прерываний от отдельных линий порта;
- 2) PxIFG (Interrupt Flag) – регистр флагов прерываний, в котором аппаратно устанавливается бит при возникновении события;
- 3) PxIES (Interrupt Edge Select) – регистр выбора фронта сигнала, по которому формируется запрос на прерывание;
- 4) PxIV (Interrupt Vector) – регистр вектора прерываний, позволяющий определить, какая из линий порта вызвала прерывание (используется при наличии нескольких активных источников в одном порту).

Взаимодействие порта с подсистемой прерываний сводится к последовательной настройке направления линии порта и подтягивающего резистора для формирования устойчивого логического уровня на входе, разрешению прерывания от нужного вывода с выбором фронта срабатывания и последующей обработке события в обработчике прерывания с обязательным сбросом соответствующего флага в регистре PxIFG.

Все аппаратные источники прерываний микроконтроллера имеют свои векторы прерываний, расположенные в специальной таблице в старших адресах памяти. Каждый вектор представляет собой 16-разрядный адрес, по которому располагается обработчик соответствующего прерывания.

Таблица 3.3 – Основные векторы прерываний MSP430F5529

Источник прерывания	Обозначение вектора	Назначение
Сброс устройства (Reset)	#pragma vector=RESET_VECTOR	Точка входа, инициализация микроконтроллера
Системные немаскируемые прерывания (SYSNMI: OFIFG, WDTIFG и др.)	#pragma vector=SYSNMI_VECTOR	Обработка сбоев генератора, WDT в режиме сброса, системных ошибок
Пользовательские немаскируемые прерывания (UNMI: VMAIFG, JTAG, PMM)	#pragma vector=UNMI_VECTOR	Обработка ошибок питания, доступа к пустой памяти, событий JTAG
Таймер B0, канал CCR0	#pragma vector=TIMER0_B0_VECTOR	Обработчик точного периодического таймера TB0CCR0
Таймер B0, каналы CCR1...CCR6 и флаг TBIFG	#pragma vector=TIMER0_B1_VECTOR	Обработчик остальных каналов TB0, захват/измерение интервалов
Сторожевой таймер WDT	#pragma vector=WDT_VECTOR	Обработчик WDT-таймера, периодические задачи/контроль зависаний
Последовательный интерфейс USCI_A0 (UART/SPI)	#pragma vector=USCI_A0_VECTOR	Приём/передача по UART/SPI на модуле USCI_A0
Последовательный интерфейс USCI_B0 (I ² C/SPI)	#pragma vector=USCI_B0_VECTOR	Обработка обмена по I ² C/SPI на модуле USCI_B0
АЦП ADC12_A	#pragma vector=ADC12_VECTOR	Обработка завершения преобразования, чтение результатов ADC12

Продолжение таблицы 3.3

Таймер A0, канал CCR0	#pragma vector=TIMER0_A0_VECTOR	Обработчик таймера TA0CCR0 (периодический тик, ШИМ и т.п.)
Таймер A0, каналы CCR1...CCR4 и флаг TAIFG	#pragma vector=TIMER0_A1_VECTOR	Обработка дополнительных каналов TA0, событий захвата/сравнения
Подсистема питания / LDO (LDO PWR)	#pragma vector=LDO_PWR_VECTOR	Реакция на события подсистемы питания и LDO-регулятора
Контроллер DMA	#pragma vector=DMA_VECTOR	Обработка завершения передач DMA, подготовка/анализ данных
Таймер A1, канал CCR0	#pragma vector=TIMER1_A0_VECTOR	Периодический таймер на TA1CCR0
Таймер A1, каналы CCR1...CCR2 и флаг TA1IFG	#pragma vector=TIMER1_A1_VECTOR	Обработка дополнительных каналов TA1
Порт ввода-вывода 1 (P1.x)	#pragma vector=PORT1_VECTOR	Обработка прерываний от линий порта P1 (кнопки, датчики)
Последовательный интерфейс USCI_A1 (UART/SPI)	#pragma vector=USCI_A1_VECTOR	Приём/передача по второму UART/SPI (модуль USCI_A1)
Последовательный интерфейс USCI_B1 (I ² C/SPI)	#pragma vector=USCI_B1_VECTOR	Обработка обмена по второй шине I ² C/SPI (USCI_B1)
Таймер A2, каналы CCR1...CCR4 и флаг TA2IFG	#pragma vector=TIMER2_A1_VECTOR	Обработка дополнительных каналов TA2
Порт ввода-вывода 2 (P2.x)	#pragma vector=PORT2_VECTOR	Обработка прерываний от линий порта P2
Таймер реального времени RTC_A	#pragma vector=RTC_VECTOR	Обработка событий RTC

4. ВЫПОЛНЕНИЕ РАБОТЫ

4.1 Описание исходного текста программы

Для решения задачи разработан алгоритм, который обновляет состояния светодиода, и обрабатывает нажатия кнопки с помощью прерываний. После обработки прерывания кнопки, запускается алгоритм для обработки дребезга контактов и далее изменяет режим работы светодиодов, после отпускания кнопки. Описание исходного текста программы:

В строках 7-14 задаются константы для различных аспектов работы программы: количество проверок для устранения дребезга кнопки, режимы работы светодиодов (только красный, только желтый, оба светодиода), состояния кнопки (не нажата, нажата) и задержка между переключениями светодиодов.

В строках 16-17 объявляются дополнительные константы для антидребезгового алгоритма: задержка между проверками кнопки и минимальное количество успешных проверок для определения нажатия.

В строках 19-20 определяются глобальные переменные: текущий режим работы светодиодов (`mode`) и предыдущее состояние кнопки (`btn_prev_state`), которые используются в логике программы.

В строках 22-27 приведены прототипы функций, которые используются для реализации задержки, устранения дребезга, переключения режимов и управления состоянием светодиодов.

В строках 28-47 реализована основная функция программы `main`. В строке 30 отключается сторожевой таймер. В строках 31-35 настраиваются порты для управления светодиодами и кнопкой: порты `P1.0` и `P8.1` устанавливаются как выходы, а порт `P1.7` — как вход с подтягивающим резистором. В строках 36-38 включаются прерывания для кнопки, настраивается срабатывание по заднему фронту и сбрасывается флаг прерывания. В строках 39-40 оба светодиода включаются в начальном состоянии. В строке 41 включаются глобальные прерывания. Основным циклом программы в строках 42-46 управляет переключением светодиодов, если активен один из режимов (красный или желтый), с использованием функции `toggle_current_mode` и заданной задержки.

В строках 49-54 реализована функция `delay`, создающая программную задержку на основе цикла.

В строках 56-61 реализована функция `next_mode`, которая переключает режим работы светодиодов на следующий (циклически) и выключает оба светодиода перед изменением режима.

В строках 63-80 реализована функция `toggle_current_mode`, которая переключает состояние светодиодов в зависимости от текущего режима. В режиме `RED_MODE` изменяется состояние красного светодиода, в `YELLOW_MODE` — желтого, а в `BOTH_MODE` оба светодиода включаются.

В строках 83-109 реализован обработчик прерываний для порта P1. В строках 87-88 проверяется, вызвано ли прерывание нажатием кнопки на порту P1.7. В строках 90-94 обрабатывается переход на следующий режим работы светодиодов при срабатывании прерывания на переднем фронте. В строках 95-103 выполняется антидребезговая обработка, которая проверяет состояние кнопки в цикле с задержкой, чтобы отсеять ложные срабатывания. Если количество проверок, фиксирующих нажатие кнопки, достигает заданного порога, настраивается новое прерывание на задний фронт. В строке 107 сбрасывается флаг прерывания для порта P1.

Каждый бит в регистрах P1IE, P1IFG и P1IES соответствует конкретному выводу порта. В данной работе используется линия P1.7, к которой подключена кнопка. Разрешение прерывания от этой линии выполняется командой P1IE |= BIT7; прерывание формируется по спаду сигнала, то есть при переходе уровня с логической «1» на «0» (что соответствует нажатию кнопки при подтяжке к Vcc), а при выполнении команды P1IES &= ~BIT7; прерывание формируется по фронту сигнала, то есть при переходе с «0» на «1» (отпускание кнопки). Флаг прерывания устанавливается аппаратно в регистре P1IFG при возникновении выбранного события, и в обработчике прерывания этот флаг необходимо обязательно сбросить программно, например командой P1IFG &= ~BIT7;,, иначе обработчик будет вызываться повторно.

4.2 Исходный текст программы

```

001 // гр. 250541 Власов Роман Евгеньевич
002 // Лабораторная работа №2
003 // Тема: «Подсистема прерываний микроконтроллера MSP430F5529»
004
005 #include <msp430.h>
006
007 #define NUM_CHECKS 10 // Количество проверок состояния кнопки для
    подавления дребезга
008 #define RED_MODE 0 // Режим, в котором мигает только красный
    светодиод
009 #define YELLOW_MODE 1 // Режим, в котором мигает только желтый
    светодиод
010 #define BOTH_MODE 2 // Режим, в котором оба светодиода горят
    постоянно
011 #define NUM_MODES 3 // Общее количество режимов
012 #define INACTIVE 0 // Состояние кнопки: не нажата
013 #define ACTIVE 1 // Состояние кнопки: нажата
014 #define TOGGLE_DELAY 10000 // Задержка между переключениями
    светодиодов
015
016 const unsigned int debounce_delay = 15; // Задержка для подавления
    дребезга кнопки
017 const unsigned int min_valid_clicks = NUM_CHECKS * 70 / 100; //
    Минимальное количество проверок для определения нажатия
018
019 unsigned int mode = BOTH_MODE; // Текущий режим работы
020 unsigned int btn_prev_state = INACTIVE; // Предыдущее состояние кнопки
021
022 // Прототипы функций

```

```

023 void delay(unsigned int cycles);          // Функция задержки на основе
цикла
024 unsigned char debounce_button(void);      // Функция обработки дребезга
кнопки
025 void toggle_current_mode(void);           // Функция переключения текущего
состояния светодиодов
026 void next_mode(void);                     // Функция переключения в
следующий режим работы
027
028 int main(void)
029 {
030     WDTCTL = WDTPW | WDTHOLD; // Отключение сторожевого таймера
031     P1DIR |= BIT0; // Настройка порта P1.0 как выход (для красного
светодиода)
032     P8DIR |= BIT1; // Настройка порта P8.1 как выход (для желтого
светодиода)
033     P1DIR &= ~BIT7; // Настройка порта P1.7 как вход (для кнопки)
034     P1REN |= BIT7; // Включение подтягивающего резистора для порта P1.7
035     P1OUT |= BIT7; // Установка подтяжки кнопки к Vcc
036     P1IE |= BIT7; // Включение прерываний по кнопке на порту P1.7
037     P1IES |= BIT7; // Настройка прерывания по заднему фронту
038     P1IFG &= ~BIT7; // Сброс флага прерывания на порту P1.7
039     P1OUT |= BIT0; // Включение красного светодиода
040     P8OUT |= BIT1; // Включение желтого светодиода
041     __enable_interrupt(); // Включение глобальных прерываний
042     for(;;)
043     {
044         if (mode != BOTH_MODE) toggle_current_mode(); // Если не в
режиме "оба светодиода", переключение состояние текущего режима
045         delay(TOGGLE_DELAY); // Задержка между переключениями
046     }
047 }
048
049 // Функция задержки
050 void delay(unsigned int cycles)
051 {
052     volatile unsigned long counter;
053     for (counter = cycles ; counter > 0; --counter);
054 }
055
056 // Функция переключения на следующий режим работы
057 void next_mode(void) {
058     mode = ++mode % NUM_MODES; // Переход на следующий режим
(циклический)
059     P1OUT &= ~BIT0; // Выключение красного светодиода
060     P8OUT &= ~BIT1; // Выключение желтого светодиода
061 }
062
063 // Функция переключения состояния светодиодов в текущем режиме
064 void toggle_current_mode(void)
065 {
066     switch (mode)
067     {
068         case RED_MODE:
069             P1OUT ^= BIT0; // Переключение состояния красного светодиода
070             break;
071
072         case YELLOW_MODE:
073             P8OUT ^= BIT1; // Переключение состояния желтого светодиода
074             break;
075
076         case BOTH_MODE:
077             P1OUT |= BIT0; // Включение красного светодиода
078             P8OUT |= BIT1; // Включение желтого светодиода

```

```

079             break;
080         }
081     }
082
083     // Обработчик прерывания для порта P1
084     #pragma vector=PORT1_VECTOR
085     __interrupt void Port_1(void)
086     {
087         // Проверка, что прерывание произошло из-за кнопки (P1.7)
088         if (P1IFG & BIT7)
089         {
090             unsigned int i = 0;
091             if (P1IES & BIT7) {
092                 unsigned int cnt = 0;
093                 for (i = 0; i < NUM_CHECKS; ++i) { //подтверждение низкого
уровня
094                     delay(debounce_delay);
095                     if (!(P1IN & BIT7)) ++cnt; //считаем 0
096                 }
097                 if (cnt >= min_valid_clicks) {
098                     P1IES &= ~BIT7; //ожидание подъема
099                 }
100             }
101             else {
102                 //ожидание подъема и подтверждение высокого уровня
103                 unsigned int cnt = 0;
104                 for (i = 0; i < NUM_CHECKS; ++i) {
105                     delay(debounce_delay);
106                     if (P1IN & BIT7) ++cnt; //считаем 1
107                 }
108                 if (cnt >= min_valid_clicks) {
109                     next_mode(); //произошло отпущание
110                     if (mode == BOTH_MODE) toggle_current_mode();
111                     P1IES |= BIT7; //ожидание падения
112                 }
113             }
114         }
115
116         P1IFG &= ~BIT7; // Сброс флага прерывания
117     }
118 }

```

5. ВЫВОДЫ

В ходе выполнения лабораторной работы были изучены принципы работы подсистемы прерываний микроконтроллера MSP430F5529, включая типы прерываний, организацию таблицы векторов и механизмы приоритезации запросов. На практике был освоен порядок конфигурирования прерываний от портов ввода-вывода: настройка линии P1.7 в качестве входа с подтягивающим резистором, разрешение прерывания, выбор фронта срабатывания, работа с флагами P1IE, P1IES, P1IFG и описанием обработчика с использованием вектора PORT1_VECTOR.

Была разработана программа управления двумя светодиодами и кнопкой, в которой смена режимов индикации осуществляется на основе прерываний, а программная обработка дребезга контактов встроена в алгоритм работы обработчика.